

GML — FOLHA 1: FUNDAMENTOS

Sintaxe atual do GameMaker (GML 2.3+ · runtime 2024+)

Variáveis · Operadores · Fluxo · Funções · Matemática

01

01 VARIÁVEIS

02 OPERADORES

03 CONDIÇÕES

04 LAÇOS

05 FUNÇÕES

06 MATEMÁTICA

07 MACROS/ENUMS

08 ORGANIZAÇÃO

09 ERROS

01 VARIÁVEIS E TIPOS DE DADOS

GML É DINAMICAMENTE TIPADA — O TIPO VEM DO VALOR

COMANDO / CONCEITO	O QUE FAZ	EXEMPLO GML	DICA PRÁTICA
real	Todo número é real de 64 bits, inteiro ou fracionário.	vel = 4.5; vidas = 3;	Não há tipo "int" separado para o dia a dia.
string	Texto entre aspas duplas; \n quebra linha.	txt = "HP: " + string(hp);	+ só junta string com string → string() .
bool	Constantes true / false (valem 1 e 0).	vivo = true; if (!vivo) exit;	Valem 1/0: dir = direita - esquerda .
array	Lista ordenada; o índice começa em 0.	itens = ["espada", "escudo"];	Último: itens[array_length(itens) - 1]
struct	Agrupa pares nome: valor numa variável.	stats = { hp: 10, atk: 2 };	Acesso: stats.hp += 5; (Folha 3 §01).
undefined	"Sem valor": função sem return, índice vazio.	if (is_undefined(r)) r = 0;	noone (-4) = "nenhuma instância".
typeof()	Devolve o nome do tipo em tempo de execução.	typeof(3) // "number"	Testes: is_string is_real is_array is_struct
ESCOPO — onde a variável existe e por quanto tempo			
= (atribuir)	Cria a variável, se preciso, e guarda o valor.	pontos = 0;	Ler antes de atribuir = erro de execução.
var (local)	Existe só no evento ou função atual.	var dx = x - xprevious;	Mais rápida; ideal para contas temporárias.
de instância	Cada instância tem a sua. Crie no Create.	hp = 100; // Create	De fora: obj_player.hp ou inst.hp
global.	Uma só para o jogo inteiro, entre rooms.	global.moedas = 0;	Inicialize uma única vez (script/controlador).

02 OPERADORES

OPERADOR	O QUE FAZ	EXEMPLO GML	DICA
+ - * /	Aritmética; / sempre dá real.	m = (a + b) / 2;	7 / 2 → 3.5
div	Divisão inteira (sem resto).	cel = x div 32;	pixel → célula
mod %	Resto da divisão.	par = (n mod 2 == 0);	ciclos, par/impar
== !=	Igual / diferente → bool.	if (hp == 0) morrer();	compare com ==
< > <= >=	Menor / maior (ou igual).	if (x > 640) x = 0;	reais: evite ==
=	Atribuição.	vel = 3;	if (a = b): evite
&& !	E, OU, NÃO (and or not).	if (vivo && !pausa) {}	^^ = xor lógico
++ --	Soma / subtrai 1.	vidas--; i++;	use isolado
+= -= *= /=	Opera e atribui.	hp -= dano; v *= 0.9;	*= 0.9 freia
? :	Ternário: cond ? sim : não	s = no_chao ? 1 : 0;	if curto
?? ??=	Troca undefined	n = valor ?? 0;	valor padrão

04 LAÇOS DE REPETIÇÃO

COMANDO	O QUE FAZ	QUANDO USAR
for	Contador: início; condição; passo.	arrays, grades, N itens
while	Repete enquanto for verdadeiro.	Δ condição precisa mudar!
repeat (n)	Repete exatamente n vezes, sem contador.	criar vários objetos
do...until	Roda 1+ vezes; para quando for verdadeiro.	sortear até ser válido
break	Sai do laço imediatamente.	achou o que buscava
continue	Pula para a próxima iteração.	ignorar itens inválidos

```
var total = 0; // for
for (var i = 0; i < array_length(loja); i++) {
    if (loja[i].vendido) continue;
    total += loja[i].preco;
}
while (place_meeting(x, y, obj_bloco)) y -= 1; // while
repeat (8) instance_create_layer(x, y, "Instances", obj_moeda);
var px; do { px = irandom(room_width); } // do...until
until (!place_meeting(px, y, obj_bloco));
```

05 FUNÇÕES

SCRIPTS · MÉTODOS

CONCEITO	O QUE FAZ	EXEMPLO GML
function	Declara; num Script é global.	function dobro(_n) { return _n * 2; }
argumentos	Parâmetros, com valor padrão.	function curar(_q = 1) { hp += _q; }
return	Devolve valor e encerra.	return clamp(_v, 0, 1);
var	Local à chamada da função.	var soma = 0;
método	Função guardada em variável.	atacar = function() { hp--; };
static	Mantém valor entre chamadas.	static usos = 0; usos++;

Script: scr_util

```
/// @function aproximar(valor, alvo, passo)
/// @description Leva "valor" até "alvo" sem ultrapassar.
function aproximar(_valor, _alvo, _passo) {
    if (_valor < _alvo) return min(valor + _passo, _alvo);
    return max(_valor - _passo, _alvo);
}
hsp = aproximar(hsp, 0, 0.3); // em qualquer objeto: freia
```

🔍 DICA

Prefixe parâmetros com _ para não confundir com variáveis da instância. Sem return, a função devolve undefined.

09 ERROS COMUNS DE INICIANTE

X ERRADO → ✓ CORRETO

X ERRADO	✓ CORRETO	POR QUÊ
txt = "Pontos: " + pontos;	txt = "Pontos: " + string(pontos);	Somar string com número gera erro de execução.
hp -= 1; // hp nunca criado	hp = 3; // no evento Create	"Variable ... not set before reading it": crie antes de ler.
draw_text(8, 8, hp); // no Step	draw_text(8, 8, hp); // no Draw GUI	draw_* só funciona em eventos Draw. Com Draw próprio, chame draw_self() .
keyboard_check(ord("w"))	keyboard_check(ord("W"))	ord() para teclas exige letra maiúscula.
for (i = 0; i < 10; i++)	for (var i = 0; i < 10; i++)	Sem var, i viri variável permanente da instância.
#macro VEL = 4;	#macro VEL 4	O = e o ; virariam parte do valor da macro.
u = itens[array_length(itens)];	u = itens[array_length(itens) - 1];	Índices vão de 0 a tamanho - 1; fora disso é erro.

03 CONDIÇÕES

COMANDO	O QUE FAZ	EXEMPLO GML	DICA
if	Executa se for verdadeiro.	if (hp <= 0) morrer();	sempre com ()
else · else if	Alternativa / nova condição.	else if (hp < 20) {}	só 1 ramo roda
switch·case	Compara 1 valor com vários.	case ARMA.ARCO:	número, string, enum
default·break	Nenhum case · sai do switch.	default: break;	sem break, "cai"

EXEMPLO DE JOGO

cor da vida + próxima fase

```
if (hp > hp_max * 0.5) cor = c_white;
else if (hp > hp_max * 0.2) cor = c_gray;
else cor = c_dkgray;
switch (room) {
    case rm_fase1: proxima = rm_fase2; break;
    case rm_fase2: proxima = rm_chefe; break;
    default: proxima = rm_menu; break;
}
```

06 MATEMÁTICA E ALEATORIEDADE

abs(-5)	5	random(10)	real 0 ≤ n < 10
min(3, 8, 1)	1	random_range(2, 5)	real 2 ... 5
max(3, 8, 1)	8	irandom(6)	int 0 ... 6
clamp(12, 0, 10)	10	irandom_range(1, 6)	int 1 ... 6
round(2.7)	3	choose("a", "b", "c")	1 dos valores
floor(2.7)	2	dsin(90) · dcos(0)	1 · 1 (graus)
ceil(2.1)	3	point_distance(x1,y1,x2,y2)	px
sign(-3)	-1	point_direction(x1,y1,x2,y2)	graus
sqr(16)	4	lengthdir_x(dist, dir)	Δx
power(2, 3) · sqr(4)	8 · 16	lengthdir_y(dist, dir)	Δy
lerp(0, 10, 0.25)	2.5	angle_difference(a, b)	-180 ... 180

ATIRAR NA DIREÇÃO DO MOUSE

obj_player · Step

```
var dir = point_direction(x, y, mouse_x, mouse_y);
var bx = x + lengthdir_x(16, dir); // ponta da arma
var by = y + lengthdir_y(16, dir);
var b = instance_create_layer(bx, by, "Instances", obj_bala);
b.direction = dir; b.speed = 8; b.image_angle = dir;
```

DEFINIÇÃO

Ângulos são em graus, anti-horário: 0° direita, 90° cima, 180° esquerda, 270° baixo. O y cresce para baixo (lengthdir_y já compensa). Use randomize() 1× no início.

07 CONSTANTES: #MACRO E ENUM

#macro TILE 32

#macro GRAVIDADE 0.4

#macro VERSAO "1.0.2"

vsp += GRAVIDADE;

var col = x div TILE;

enum ARMA { ESPADA, ARCO, CAJADO }

// valores 0, 1, 2

enum PRIO { BAIXA = 1, ALTA = 10 }

arma = ARMA.ARCO;

if (arma == ARMA.ARCO) atirar();

#macro: trocada na compilação, sem = e sem ; .

enum: inteiros constantes (ENUM.NOME). Ambos globais.

08 COMENTÁRIOS E ORGANIZAÇÃO

// comentário de linha

/* comentário de bloco */

/// @description Doc JS Doc

#region Movimento

x += hsp;

#endregion

■ Prefixos: obj_ spr_ snd_ rm_ scr_ fnt_

■ Variáveis em snake_case; constantes em MAIÚSCULAS .

■ Create: variáveis · Step: lógica · Draw: só desenho.

■ Nomes que explicam: vel_max, não v2 .

■ #region dobra blocos longos no editor.

GML — FOLHA 2: GAMEPLAY

Construindo o jogo · eventos, entrada, física e sistemas
obj_bloco = objeto sólido de parede/chão usado nos exemplos

02

01 EVENTOS 02 TECLADO 03 MOUSE 04 MOVIMENTO 05 COLISÃO 06 PLATAFORMA 07 SPRITES 08 INSTÂNCIAS 09 ROOMS 10 CÂMERA 11 JOGADOR 12 TIMERS 13 DADOS 14 ÁUDIO 15 PARTÍCULAS

02-05 ENTRADA, MOVIMENTO E COLISÃO

TABELA PRINCIPAL · K = VK_* OU ORD("LETRA") · MB = MB_LEFT / MB_RIGHT / MB_MIDDLE

COMANDO	FUNÇÃO	EXEMPLO	DICA
02 · TECLADO exemplo abaixo: WASD + setas, pulo e ataque			
keyboard_check	Tecla segurada.	keyboard_check(vk_right)	andar
keyboard_check_pressed	Apertou neste frame.	__pressed(vk_space)	pular
keyboard_check_released	Soltou neste frame.	__released(ord("X"))	carregar
keyboard_check_direct	Estado físico da tecla.	__direct(vk_f1)	só Windows
ord("W")	Código de letra/dígito.	keyboard_check(ord("W"))	MAIÚSCULA
vk_*	Teclas especiais.	vk_space vk_shift	vk_anykey

```
var h = (keyboard_check(ord("D")) || keyboard_check(vk_right))
- (keyboard_check(ord("A")) || keyboard_check(vk_left));
var v = (keyboard_check(ord("S")) || keyboard_check(vk_down))
- (keyboard_check(ord("W")) || keyboard_check(vk_up));
if (keyboard_check_pressed(vk_space)) pular();
if (keyboard_check_pressed(ord("J")) && recarga == 0) atacar();
```

COMANDO	FUNÇÃO	EXEMPLO	DICA
04 · MOVIMENTO			
x · y	Posição em px.	x += 2;	y ↓
hspeed · vspeed	Velocidade h / v.	hspeed = -4;	auto
speed · direction	Velocidade + ângulo.	direction = 90;	tiros
friction	Freia speed.	friction = 0.2;	deslizar
motion_add(dir, spd)	Soma um vetor.	motion_add(270, 0.3);	impulso
motion_set(dir, spd)	Define o vetor.	motion_set(45, 4);	lançar
05 · COLISÃO retornam id ou noone · prec = precisa · notme = ignora a si			
place_meeting	Colidiria em x, y?	(x, y + 1, obj_bloco)	antes
place_free	Livre de Solid?	(x, y + 1)	só Solid
move_and_collide 2024+	Move e desliza.	(hsp, vsp, obj_bloco)	→ array
instance_place	Quem colidiria.	(x, y, obj_inimigo)	agir
instance_position	Instância num ponto.	(mouse_x, mouse_y, obj_botao)	clique
collision_rectangle	Algo no retângulo?	(x1,y1,x2,y2, obj, false, true)	hitbox
collision_circle	Algo no círculo?	(x, y, raio, obj, false, true)	visão
collision_point	Algo no ponto?	(x, y, obj, prec, notme)	sensor

01 EVENTOS

EVENTO	USO TÍPICO
Create	1× ao nascer: iniciar variáveis.
Begin Step	Início do frame: ler entrada.
Step	Todo frame: lógica, movimento, IA.
End Step	Após colisões: câmera, ajustes.
Draw	Desenho na room; use draw_self().
Draw GUI	HUD, em coordenadas de tela.
Alarm 0-11	Quando alarm[n] chega a 0.
Keyboard+Mouse	Tecla / clique (ou Global).
Collision	Ao tocar; other = o outro.
Room Start+End	Entrar / sair da room.
Clean Up	Liberar ds_*, surfaces etc.

B.Step · Alarm · Input · Step · move · Colisão · E.Step · Draw

07 SPRITES E ANIMAÇÃO

sprite_index	sprite atual
image_index	quadro atual (0 = 1º)
image_speed	ritmo da animação (0 = parada)
image_xscale · _yscale	escala: -1 espelha
image_angle	rotação em graus
image_alpha	opacidade 0 ... 1
image_blend	tinta (c_white = normal)
sprite_width · _height	tamanho em px com escala

```
if (!chao) sprite_index = spr_pula;
else if (hsp != 0) sprite_index = spr_corre;
else sprite_index = spr_parado;
if (hsp != 0) image_xscale = sign(hsp);
```

09 ROOMS

```
room_goto(rm_fase2) · ir
room_restart() · reiniciar a atual
room_goto_next() · room_goto_previous()
room_next(room) · room_previous(room) · id/-1
room_width · room_height · tamanho
```

08 INSTÂNCIAS

```
var b = instance_create_layer(x, y, "Instances",
    obj_bala, { direction: 90, speed: 6 });
if (hp <= 0) instance_destroy();
if (instance_exists(obj_chefe)) { }
var e1 = instance_find(obj_inimigo, 0);
if (instance_number(obj_moeda) == 0) vencer();
with (obj_inimigo) hp -= 1; // TODOS; other = eu
```

06 SISTEMA DE PLATAFORMA

```
// Create
hsp = 0; vsp = 0; grv = 0.4;
vel = 3; forca_pulo = -7;
// Step - horizontal + gravidade
hsp = (keyboard_check(vk_right)
- keyboard_check(vk_left)) * vel;
vsp = min(vsp + grv, 10); // queda máx.
var chao = place_meeting(x, y + 1, obj_bloco);
if (chao && keyboard_check_pressed(vk_space))
    vsp = forca_pulo; // pulo
if (vsp < 0 && !keyboard_check(vk_space))
    vsp = max(vsp, forca_pulo / 2); // curto
// parede: encosta 1 px por vez e para
if (place_meeting(x + hsp, y, obj_bloco)) {
    var sx = sign(hsp);
    while (!place_meeting(x + sx, y, obj_bloco))
        x += sx;
    hsp = 0;
}
x += hsp;
if (place_meeting(x, y + vsp, obj_bloco)) { // y
    var sy = sign(vsp);
    while (!place_meeting(x, y + sy, obj_bloco))
        y += sy;
    vsp = 0;
}
y += vsp;
```

DEFINIÇÃO Direto: você soma a x/ y (controle total, como acima). Por velocidade: define speed/hspeed e o GameMaker move sozinho — ideal para projéteis. Não misture.

13 ARRAYS E STRUCTS NO JOGO

```
stats = { hp: 10, atk: 3, def: 1, vel: 2.5 };
armas = [ { nome: "Espada", dano: 4 },
    { nome: "Arco", dano: 2 } ];
var dano = armas[arma].dano + stats.atk;
mochila = []; // inventário
array_push(mochila, { nome: "Poção", qtd: 3 });
global.inimigos = {
    slime: { hp: 3, vel: 1, xp: 5 },
    morcego: { hp: 2, vel: 2.5, xp: 8 }
};
var d = global.inimigos[$ "slime"];
```

14 ÁUDIO

(SOM, PRIORIDADE, LOOP)

```
audio_play_sound(snd_pulo, 5, false); // efeito
musica = audio_play_sound(snd_tema, 10, true);
audio_sound_gain(musica, 0, 2000); // fade 2 s
audio_stop_sound(musica); audio_stop_all();
audio_master_gain(0.5); // volume geral
if (!audio_is_playing(snd_tema)) { }
audio_sound_pitch(som, random_range(0.9, 1.1));
```

11 SISTEMAS DO JOGADOR

```
// Create
hp_max = 5; hp = hp_max; inv = 0;
global.cp_x = x; global.cp_y = y;
tomar_dano = function(_dano) {
    if (inv > 0 || hp <= 0) return;
    hp = max(hp - _dano, 0);
    inv = 60; // 1 s invencível
    if (hp == 0) { // morte
        visible = false; alarm[1] = 90;
    }
};
// Step - pisca se invencível
if (inv > 0) inv--;
image_alpha = (inv mod 8 < 4) ? 1 : 0.3;
// Alarm 1 - respawn no checkpoint
x = global.cp_x; y = global.cp_y;
hp = hp_max; vsp = 0; inv = 90; visible = true;
// obj_espinho · Collision com obj_player
other.tomar_dano(1);
// obj_checkpoint · Collision com obj_player
global.cp_x = x; global.cp_y = y;
```

10 CÂMERA

ATIVE O VIEWPORT 0

```
var alvo = obj_player; // End Step
var cam = view_camera[0];
var w = camera_get_view_width(cam);
var h = camera_get_view_height(cam);
var tx = clamp(alvo.x - w/2, 0, room_width - w);
var ty = clamp(alvo.y - h/2, 0, room_height - h);
camera_set_view_pos(cam,
    lerp(camera_get_view_x(cam), tx, 0.1),
    lerp(camera_get_view_y(cam), ty, 0.1));
camera_set_view_size(cam, 480, 270); // zoom
```

12 TIMERS

ALARM -1/FRAME · -1 = OFF

```
alarm[0] = game_get_speed(gamespeed_fps) * 3;
if (recarga > 0) recarga--; // contador
var dt = delta_time / 1000000; // ps - s
x += 120 * dt; // 120 px/s
call_later(2, time_source_units_seconds, abrir);
// ↑ chama abrir() após 2 s (versões 2024+)
```

15 PARTÍCULAS

SEM CRIAR INSTÂNCIAS

```
ps = part_system_create(); // Create
pt = part_type_create();
part_type_shape(pt, pt_shape_pixel);
part_type_colour2(pt, c_white, c_gray);
part_type_alpha2(pt, 1, 0);
part_type_speed(pt, 1, 3, -0.05, 0);
part_type_direction(pt, 0, 359, 0, 0);
part_type_life(pt, 20, 40); // frames
part_particles.create(ps, x, y, pt, 20); // uso
part_type_destroy(pt); part_system_destroy(ps);
```

★ GAMEMAKER SYSTEMS — REFERÊNCIA RÁPIDA

PADRÕES PRONTOS PARA ADAPTAR

JOGADOR Step

```
var h = keyboard_check(ord("D"))
- keyboard_check(ord("A"));
move_and_collide(h * 3, 0, obj_bloco);
if (h != 0) image_xscale = sign(h);
```

INIMIGO (PATRULHA) Create: dir = 1; · Step

```
if (place_meeting(x + dir, y, obj_bloco)
|| !place_meeting(x + dir * 8, y + 1, obj_bloco))
    dir = -dir; // parede ou beirada
x += dir * 1.5; image_xscale = dir;
```

PROJÉTIL Create: speed = 8; dano = 1;

```
// Step
if (place_meeting(x, y, obj_bloco)) instance_destroy();
// Collision com obj_inimigo
other.hp -= dano; instance_destroy();
```

COLETÁVEL Collision com obj_player

```
global.moedas += 1;
audio_play_sound(snd_moeda, 5, false);
instance_destroy();
```

PORTA Collision c/ obj_player

```
if (keyboard_check_pressed(ord("E"))) {
    room_goto(destino); // Variable Definition
}
```

CHECKPOINT Collision c/ obj_player

```
if (!ativo) {
    ativo = true; image_index = 1; // aceso
    global.cp_x = x; global.cp_y = y;
}
```

GML — FOLHA 3: AVANÇADO + REFERÊNCIA

01 STRUCTS02 ARRAYS03 DS_*04 ENUMS05 ESTADOS06 FUNÇÕES07 SAVE08 BUFFERS09 SURFACES10 LAYERS11 PATHFINDING12 DEBUG13 DESEMPENHO14 ERROS15 REFERÊNCIA16 DESAFIOS

01 STRUCTSREFERÊNCIA · SEM DESTROY

```

var heroi = { nome: "Ayla", hp: 10 }; // criar heroi.hp -= 2; heroi["$mana"] = 5; // + membro
function Item(_nome, _preco) constructor {
    nome = _nome; preco = _preco;
    static descrever = function() { // método
        return nome + ": $" + string(preco);
    };
}
function Arma(_n, _p, _dano) : Item(_n, _p)
    constructor { dano = _dano; } // herança
var espada = new Arma("Espada", 50, 4);
espada.descrever(); // herdado

```

12 DEPURAÇÃOF6 DEBUG · F9 BREAKPOINT

```

show_debug_message("hp = " + string(hp));
show_debug_overlay(true); // FPS

```

05 MÁQUINA DE ESTADOS DO JOGADOR

```

// Create
estado = ESTADO.IDLE; t_dano = 0;
sprites = [spr_idle, spr_walk, spr_jump, spr_fall, spr_atk, spr_hurt, spr_dead];
// Step
var h = keyboard_check(vk_right)
    - keyboard_check(vk_left);
var chao = place_meeting(x, y + 1, obj_bloco);
var pulo = keyboard_check_pressed(vk_space);
switch (estado) {
    case ESTADO.IDLE:
    case ESTADO.WALK:
        hsp = h * 3;
        estado = h != 0 ? ESTADO.WALK : ESTADO.IDLE;
}

```

07 SALVAR E CARREGARINI · JSON

```

ini_open("config.ini"); // INI
ini_write_real("audio", "volume", global.vol);
global.vol = ini_read_real("audio", "volume", 1);
ini_close(); // feche!
function salvar(_arq, _dados) { // JSON
    var s = json_stringify(_dados);
    var b = buffer_create(1, buffer_grow, 1);
    buffer_write(b, buffer_string, s);
    buffer_save(b, _arq); buffer_delete(b);
}
function carregar(_arq) {
    if (!file_exists(_arq)) return undefined;
    var b = buffer_load(_arq);
    var s = buffer_read(b, buffer_string);
    buffer_delete(b); return json_parse(s);
}

```

08 BUFFERSMEMÓRIA BINÁRIA TIPADA

AVANÇADO
Arquivos, rede, saves · leia na mesma ordem.

```

var b = buffer_create(16, buffer_grow, 1);
buffer_write(b, buffer_u8, vidas); // 1 byte
buffer_seek(b, buffer_seek_start, 0); // início
vidas = buffer_read(b, buffer_u8);
buffer_delete(b); // libere

```

13 CHECKLIST DE DESEMPENHO

- ☐ Destrua o que sai da tela; evite criar instâncias em massa.
- ☐ Sem laços aninhados sobre todas as instâncias no Step.
- ☐ Colisão só quando precisa; máscaras simples.
- ☐ Todo ds_*_create tem seu destroy.
- ☐ Surfaces e partículas: crie 1x, reutilize, libere.
- ☐ Arrays grandes: array_create(n); cache o tamanho.
- ☐ Cálculos repetidos e temporários em var.
- ☐ Objetos-pai + event_inherited(); meça com o Profiler.

15 REFERÊNCIA RÁPIDA DE GML

ENTRADA	MOVIMENTO	COLISÃO	SPRITES	INSTÂNCIAS	ROOMS	CÂMERA
keyboard_check keyboard_check_pressed mouse_check_button mouse_x · mouse_y	hspeed · vspeed speed · direction motion_add · _set move_towards_point	place_meeting instance_place move_and_collide collision_rectangle	sprite_index image_index image_xscale draw_self()	instance_create_layer instance_destroy instance_exists instance_nearest	room_goto room_restart room_goto_next room_width · _height	view_camera[0] camera_get_view_x camera_set_view_pos camera_set_view_size
AÚDIO	MATEMÁTICA	ARRAYS	STRUCTS	ESTRUT. DE DADOS	DEPURAÇÃO	DESENHO
audio_play_sound audio_stop_sound audio_is_playing audio_sound_gain	clamp · lerp point_distance point_direction irandom_range	array_length array_push · _pop array_insert array_delete	s.a · s[\$ "a"] new Construtor() struct_get_names struct_exists	ds_list_create ds_map_create ds_grid_create ds_exists(id, tipo)	show_debug_message show_debug_overlay debug_get_callstack typeof(valor)	draw_text draw_set_font draw_set_colour draw_rectangle

16 10 DESAFIOS PROGRESSIVOS

SEM SOLUÇÕES · RESOLVA EM ORDEM

01 MOVIMENTO Mover o jogador em 8 direções. ☐ WASD e setas; diagonal normalizada ☐ Não sair dos limites da room	02 PULO Gravidade e pulo de plataforma. ☐ Pular só no chão ☐ Pulo curto e queda máxima	03 COLISÃO Paredes, chão e teto sólidos. ☐ Sem atravessar nem tremer ☐ Parar rente à parede	04 INIMIGO Patrulhar e perseguir. ☐ Vira em paredes e beiradas ☐ Persegue a menos de 120 px	05 HP E DANO Vida com invencibilidade. ☐ Barra de HP no Draw GUI ☐ Piscar 1 s; respawn
06 INVENTÁRIO Coletar e usar itens. ☐ Array de structs ☐ Empilhar itens iguais	07 TIRO Atirar na direção do mouse. ☐ Recarga entre disparos ☐ Dano e destruição ao colidir	08 ESTADOS Máquina de estados completa. ☐ Enum com os 7 estados ☐ Um sprite por estado	09 SAVE/LOAD Persistir o progresso. ☐ HP, fase e itens em JSON ☐ Tratar arquivo ausente	10 MINI JOGO Uma fase completa e jogável. ☐ Unir os desafios 1–9 ☐ Vitória, derrota e porta

02 ARRAYS

```

var a = [10, 20, 30];
array_push(a, 40); // [10,20,30,40]
var u = array_pop(a); // 40 -> [10,20,30]
var p = array_shift(a); // 10 -> [20,30] 2024+
array_insert(a, 1, 25); // [20,25,30]
array_delete(a, 0, 1); // (índice, qtd) [25,30]
var n = array_length(a); // 2
var m = [[0, 1], [1, 0]]; m[1][0] = 2; // 2D

```

04 ENUMS PRÁTICOS

```

enum ESTADO { IDLE, WALK, JUMP, FALL, ATTACK, HURT, DEAD }
enum ARMA { ESPADA, ARCO, BOMBA, TOTAL }
enum INIMIGO { SLIME, MORCEGO, CHEFE }
enum JOGO { MENU, JOGANDO, PAUSA, FIM }
arma = (arma + 1) mod ARMA.TOTAL; // ciclo

```

IDLE≠WALK -> JUMP -> FALL -> IDLE · ATTACK -> IDLE · HURT -> IDLE/DEAD

```

if (!chao) estado = ESTADO.FALL;
if (pulo) { vsp = -7; estado = ESTADO.JUMP; }
if (keyboard_check_pressed(ord("J"))) {
    estado = ESTADO.ATTACK;
    image_index = 0;
}
break;
case ESTADO.JUMP:
    hsp = h * 3;
    if (vsp >= 0) estado = ESTADO.FALL;
    break;
case ESTADO.FALL:
    hsp = h * 3;
    if (chao) estado = ESTADO.IDLE; break;

```

06 FUNÇÕES REUTILIZÁVEIS

```

// @param {Id.Instance} _alvo alvo do dano
// @return {Bool} true se o alvo morreu
function aplicar_dano(_alvo, _dano, _forca = 2) {
    if (!instance_exists(_alvo)) return false;
    with (_alvo) {
        hp -= _dano;
        hsp = sign(x - other.x) * _forca;
    }
    return _alvo.hp <= 0;
}

```

1 tarefa por função · verbo_objeto · _param com padrão · retorne em vez de alterar globais.

09 SURFACES E DESENHO

```

// Create: surf = -1; Draw: ... ; CU = Clean Up
if (!surface_exists(surf))
    surf = surface_create(320, 180);
surface_set_target(surf);
draw_clear_alpha(c_black, 0);
draw_circle(40, 40, 16, false);
surface_reset_target(); // sempre!
draw_surface(surf, 0, 0);
if (surface_exists(surf)) surface_free(surf); // CU

```

Luz, minimapa, rastros · voláteis · só em eventos Draw.

14 ERROS INTERMEDIÁRIOS E AVANÇADOS

✗ GML INCORRETO	✓ GML CORRETO	POR QUÊ
alarm[0] = 60; // no Step	if (alarm[0] == -1) alarm[0] = 60;	Reiniciado todo frame.
var l = ds_list_create(); // Step	lista = ds_list_create(); // Create	Vaza memória.
with (obj_inimigo) hp -= dano;	var d = dano; with (obj_inimigo) hp -= d;	dano seria do inimigo.
ax = alvo.x; // alvo destruído?	if (instance_exists(alvo)) ax = alvo.x;	Sem instância = erro.
for (var i = 0; i < n; i++)	for (var i = array_length(a) - 1; i >= 0; i--)	Ao remover, vá de trás p/ frente.
var b = a; b.hp = 0; // a muda!	var b = { hp: a.hp }; // struct novo	Structs são referências.

03 ESTRUTURAS DE DADOS

ds_list	lista	l[0] 0]
ds_map	chave -> valor	m[? "ouro"]
ds_grid	grade 2D	g[# cx, cy]
ds_queue	fila (1º a entrar sai)	_enqueue · _dequeue
ds_stack	pilha (último sai)	_push · _pop

```

lista = ds_list_create(); // Create
ds_list_add(lista, "espada"); // "espada";
var item = lista[0]; // "espada"
ds_list_destroy(lista); // Clean Up

```

▲ MEMÓRIA
ds_* não se libera sozinha: todo _create pede um _destroy no Clean Up. Verifique com ds_exists().

```

case ESTADO.ATTACK:
    hsp = 0;
    if (image_index >= image_number - 1)
        estado = ESTADO.IDLE;
    break;
case ESTADO.HURT: // t_dano = 20 ao ser atingido
    t_dano -= 1;
    if (t_dano <= 0) estado = ESTADO.IDLE;
    if (hp <= 0) estado = ESTADO.DEAD;
    break;
case ESTADO.DEAD: hsp = 0; break;
}
sprite_index = sprites[estado]; // enum = índice
// + gravidade e colisão (Folha 2 §06)

```

11 PATHFINDINGMP_GRID

```

var cw = room_width div 32; // Controlador
var ch = room_height div 32;
global.mp = mp_grid_create(0, 0, cw, ch, 32, 32);
mp_grid_add_instances(global.mp, obj_bloco, false);
rota = path_add(); alarm[0] = 1; // Inimigo
// Alarm 0 - recalcula a cada 0,5 s
if (instance_exists(obj_player)
    && mp_grid_path(global.mp, rota, x, y,
        obj_player.x, obj_player.y, true)) {
    path_start(rota, 2, path_action_stop, true);
}
alarm[0] = 30;

```

Libere com path_delete e mp_grid_destroy.

10 LAYERS

```

layer_get_id("Fundo") · id pelo nome
layer_set_visible(id, false) · layer_depth
layer_x(id, v) · layer_hspeed(id, 0.5)

instance_deactivate_layer(id) · pausa
layer_tilemap_get_id(id) -> tilemap

```

✗ INCORRETO -> ✓ CORRETO